Admissions Media Insight Engine — Product Requirements

Oct 10, 2026 · @AJ

1. Goal and users

Build a web app that turns long videos, podcasts and web pages into a searchable, timestamped knowledge base, then answers questions with links back to the exact minute or paragraph. First use: college admissions research (Yale Early Action first, then other Ivy+ schools). The design must stay domain-neutral so the same app serves other research later.

Problem today: a 35-minute admissions podcast takes 60+ minutes to listen to and take notes on. Advice on one topic (e.g., "why this major") is scattered across many episodes and pages, and it is hard to find again.

Primary user: AJ, acting as a counselor for a student applicant. **Secondary user:** the student (Aksh), read-only.

Core idea — the Lens. Before any source is analyzed, the user sets a Lens: the domain, the reader's role and what to look for. Every summary, tag and answer is produced through that Lens. Swapping the Lens (e.g., "engineering team evaluating AI tools") reuses the whole app for a new purpose with zero code changes.

Success in one sentence: I paste a list of 10 Yale essay questions and, within minutes, get a report where each question has a grounded answer plus 2–5 clickable links that open the podcast at the right timestamp.

2. Core concepts

Six nouns define the product; everything else is built from them.

Concept	What it is	Yale example
Collection	One per college, defined by one config file: its sources, its Lens and question banks	yale.yaml → "Yale"

Concept	What it is	Yale example
Lens	Free-text system context: domain, role, focus areas, tag vocabulary	"Undergrad admissions; reader is a counselor; focus on essays, testing, major fit, EA strategy"
Source	One ingested item: YouTube video, podcast episode (MP3/RSS), web page or PDF	"Inside Yale Admissions — episode on supplements"
Segment	A time slice (default 60 s) or a page section, with transcript text, summary, tags and embedding	12:00–13:00, "Writing the 'why major' short answer"
Chapter	A topic block of consecutive segments, auto-detected	09:30–17:45, "Supplemental essays"
Question Bank	A saved list of questions run in batch against a Collection	Yale supplement prompts + 10 testing/EA questions

Rule: every answer, summary and tag must trace back to one or more Segments. No Segment, no claim.

3. Functional requirements

Five capabilities, listed in build order. IDs (FR-x) are for tracking in Claude Code specs and PRs.

FR-1 Configuration files (source of truth)

Day 1 has no source-entry screen: AJ edits one YAML file per college and runs a sync. Small, reviewable, version-controlled input is the fastest route to accurate output.

- FR-1.1 One file per college in `config/collections/` (`yale.yaml` , `mit.yaml` , …). Each file becomes one Collection.
- FR-1.2 Each file holds: name, Lens (or a reference to a shared Lens file), tags, and a list of sources with URL plus optional label, type, priority and notes.
- FR-1.3 Sync via `make sync COLLECTION=yale` or a "Sync" button. It is idempotent: new URLs are ingested, unchanged URLs skipped, removed URLs archived (never hard-deleted).

- FR-1.4 Validate on load; write a sync log listing every URL as ingested, skipped or failed with reason. Nothing is skipped silently.
- FR-1.5 A shared `config/lenses/undergrad-admissions.yaml` lets every college reuse one Lens; a college file can override it.
- FR-1.6 Collections are isolated: a Yale question never cites MIT sources unless the user picks "all colleges".

```
# config/collections/yale.yaml
name: Yale
lens: undergrad-admissions        # shared Lens file
focus_note: Early Action, class of 2031
sources:
  - url: https://www.youtube.com/watch?v=XXXX
    label: Admissions podcast – supplemental essays
    priority: high
  - url: https://admissions.yale.edu/...
    label: Essay prompts page
    type: web
  - url: https://example.com/episode.mp3
    label: Podcast episode on testing
    type: audio                    # Day 2
```

FR-2 Ingest sources

- FR-2.1 Read URLs from the config file. Auto-detect type when not given: YouTube, podcast page/RSS/MP3, web page, PDF.
- FR-2.2 Get a timestamped transcript: YouTube captions first; if none, download audio and run speech-to-text. Web/PDF: extract clean text with headings as section anchors.
- FR-2.3 Capture metadata: title, channel/host, publish date, duration, URL, ingest date.
- FR-2.4 Show per-source status: queued → transcribing → analyzing → ready / failed (with reason). Jobs run in the background; the UI polls.
- FR-2.5 De-duplicate by URL.

FR-3 Per-source analysis (through the Lens)

- FR-3.1 **Overview** (one paragraph): presenter(s) and role, target audience, what the source covers, and why it matters for the Lens.
- FR-3.2 **Chapters**: topic blocks with start/end time, title and 2–3 sentence summary.

- FR-3.3 **Minute summaries**: one 1–2 sentence summary per 60 s segment (a 35-min video gives 35). Segment length is configurable (30–300 s).
- FR-3.4 **Tags** per segment from the Lens tag list (e.g., testing, essays, why-major, EA/ED, extracurriculars, recommendations, interviews, financial aid).
- FR-3.5 **Key quotes**: up to 10 notable verbatim lines per source, each with timestamp, flagged as direct speaker advice.
- FR-3.6 Source page shows overview, chapters, a minute-by-minute timeline and an embedded player. Clicking any timestamp seeks the player.

FR-4 Ask questions (grounded Q&A)

- FR-4.1 Ask a question against a whole Collection, or filter by source, tag or date range.
- FR-4.2 Answer is generated only from retrieved segments, with inline citations [1], [2].
- FR-4.3 Each citation shows source title, publish date, timestamp range, the supporting transcript snippet and a deep link (YouTube `?t=`, audio player seek, or web page anchor).
- FR-4.4 If sources do not cover the question, say so plainly ("Not covered in your sources") and list the closest segments.
- FR-4.5 If sources disagree or are older than a set age (default 18 months), flag it: admissions policy changes year to year.
- FR-4.6 Follow-up questions keep chat context. Save any answer to a Notes list.

FR-5 Question Bank and report

- FR-5.1 Paste or upload a list of questions (one per line or CSV). Save as a named bank.
- FR-5.2 Run the bank: each question goes through FR-4. Show progress.
- FR-5.3 Report view: per question — short answer, "watch these" list (2–5 timestamp links ranked by relevance), coverage indicator (strong / partial / none).
- FR-5.4 Export report to Markdown and PDF.

Non-functional

- A 35-min source is ready in under 5 minutes.
- A Q&A answer returns in under 10 seconds.
- Single user, local or one small cloud VM; basic password auth if hosted.
- API keys in env vars only; never in the repo.

- LLM prompts live in versioned files, not hard-coded, so the Lens and prompts can be tuned without redeploying.

4. Scope: Day 1 vs later

Day 1 delivers the full loop for YouTube and web pages only; podcasts via audio transcription are Day 2. That keeps the one-day build realistic.

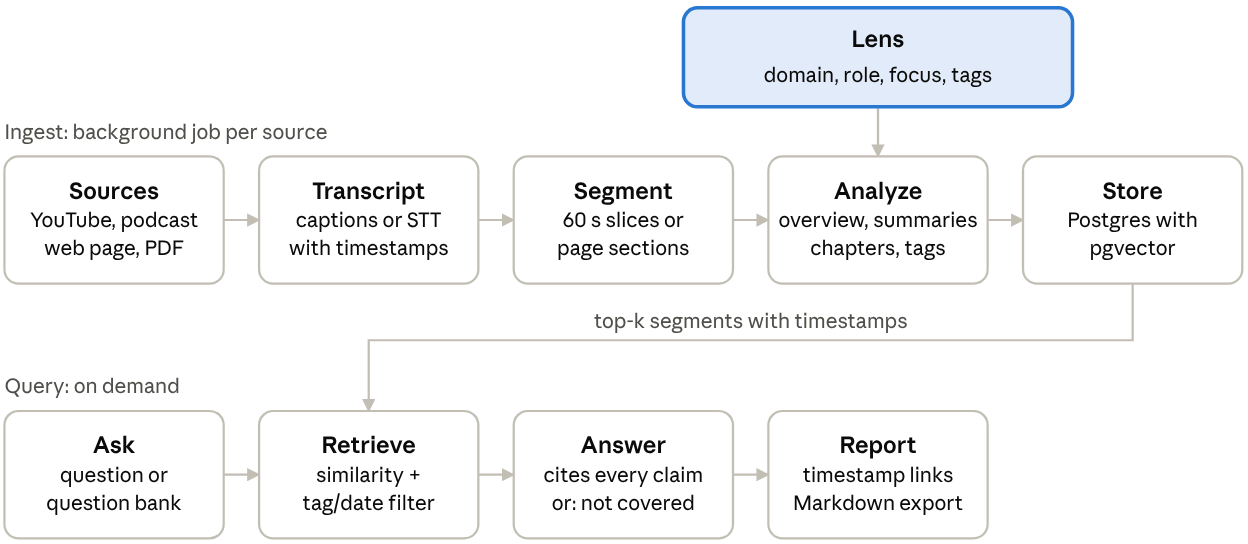
Phase	In scope	Explicitly out
Day 1 (MVP)	yale.yaml and mit.yaml + one shared Lens; sync ingests the YouTube (captions) and web page URLs listed there; overview, chapters, minute summaries, tags; source page with player seek; Q&A with timestamp citations; Question Bank run + Markdown export	Source-entry UI, audio transcription, PDF, auth, multi-user, re-analyze
Day 2	Podcast MP3/RSS + speech-to-text; PDF ingest; config files for the remaining Ivy+ schools; PDF export; staleness/conflict flags	Sharing, mobile polish
Week 2+	Re-analyze on Lens change; essay-draft checker ("does my draft address what officers said?"); scheduled ingest of new podcast episodes; share read-only view with student	—

If Day 1 runs short, cut in this order: Markdown export, tags, chapters. Never cut citations — they are the product.

5. Architecture, stack and data model

Two paths share one store: a background ingest pipeline and an on-demand question path. The engineers choose the stack; this section sets only the shape and the rules every choice must meet.

Every answer traces back to a timestamped segment



architecture · ingest pipeline and query path

The Lens shapes the Analyze step and the answer prompt; every answer is built only from segments retrieved from the store.

Constraints (the only hard rules):

- Runs with one command (e.g., Docker Compose).
- Model and cost are not constraints: use whatever answers the questions best. Spend the effort on the product.
- Every stored segment keeps `source_id`, `start_s`, `end_s` and the raw transcript text. Without these, citations break.
- Prompts live in plain files, not in code, so AJ can tune the Lens wording.
- Prefer the simplest thing that works: one process, one database, no queue, no auth on Day 1.

Keep it simple — not on Day 1: microservices, Kubernetes manifests, user accounts, a source-entry UI, fine-tuning, agent frameworks. A plain retrieve-then-answer loop is enough.

Deep links: YouTube `https://youtu.be/<id>?t=<seconds>`; podcast = in-app player seek; web page = URL + `#:~:text=<first words of snippet>`.

Data model (suggested minimum; adapt freely):

```
collections(id, name, lens_text, lens_tags text[], created_at)
sources(id, collection_id, type, url, title, host, published_at, duration_s,
status, error, overview)
chapters(id, source_id, start_s, end_s, title, summary)
segments(id, source_id, start_s, end_s, anchor, text, summary, tags text[],
embedding vector)
key_quotes(id, source_id, ts_s, quote)
question_banks(id, collection_id, name, questions text[])
answers(id, collection_id, bank_id, question, answer_md, coverage, citations
jsonb, created_at)
```

6. Work split and how to run it remotely

Agree the API contract in the first hour; after that the two engineers work in parallel against mocks and meet only at integration.

Engineer	Owns	Day-1 deliverables
Dev A (backend / Kubernetes / AI)	Config loader + sync, transcript fetch, chunking, LLM analysis prompts, embeddings, retrieval + answer API, DB schema	`make sync COLLECTION=yale` → `sources ready in < 5 min each`; `POST /ask` returns answer + citations; Docker Compose up
Dev B (frontend / Python / AI)	React UI: college picker, read-only source list + sync log, source page with player seek, Ask chat, Question Bank + report, export	All screens working against mock JSON by midday, live API by end of day

Minimum API contract (Hour 1; endpoint names are a suggestion):

- `POST /sync?collection=yale` → reads `yale.yaml`, returns the sync log; `GET /collections`
- `GET /collections/{id}/sources`; `GET /sources/{id}` (status, overview, chapters, segments)
- `POST /collections/{id}/ask` (question, filters) → answer, citations[]
- `POST /collections/{id}/question-banks/run` (questions[]) → report

Process (spec-driven with Claude Code):

1. Hour 0–1: both read this doc; Dev A writes `openapi.yaml` and `schema.sql`; Dev B writes mock JSON from it. Commit to one repo.
2. Each dev turns their FR sections into a Claude Code spec (`/specs/*.md`) before generating code.
3. Midday checkpoint: async Loom or short call; demo one source end-to-end.
4. End of day: run the acceptance tests in section 7 with a real Yale video, record a 3-minute demo for AJ.
5. Questions for AJ go in one shared thread, batched twice a day; anything not answered → pick the simplest option and note it in the README.

7. Acceptance criteria

Day 1 is done when all eleven checks pass on real Yale admissions content, demoed by video.

- [ ] AC-1 List 3 Yale YouTube URLs in yale.yaml and run sync; all reach "ready" in under 5 minutes each.
- [ ] AC-2 A 35-minute video shows 1 overview paragraph, 3–8 chapters and ~35 minute summaries, each with a working timestamp.
- [ ] AC-3 Clicking any timestamp starts the embedded player within ±5 seconds of that moment.
- [ ] AC-4 Ask "What does Yale look for in the why-this-major short answer?" → answer with at least 2 citations, each opening the right moment.
- [ ] AC-5 Ask "Is the SAT/ACT required and how much does it matter?" → answer cites the source publish date; if sources are older than 18 months the answer says to verify on the official site.
- [ ] AC-6 Ask something the sources don't cover (e.g., "What is Yale's hockey recruiting process?") → "Not covered in your sources", no invented answer.
- [ ] AC-7 Add one web page URL to yale.yaml (e.g., an admissions blog post) and re-sync → it appears in answers with a link to that page.
- [ ] AC-8 Run a 10-question bank → report lists every question with answer, coverage level and timestamp links; Markdown export opens cleanly.
- [ ] AC-9 Re-running sync with one new URL ingests only that URL; the sync log lists the rest as skipped.
- [ ] AC-10 With mit.yaml synced too, a question asked in the Yale collection never cites an MIT source.

- [ ] AC-11 The golden test set passes: at least 8 of 10 questions cite the expected source and timestamp.

Golden test set: AJ supplies 10 questions + the expected source/timestamp for each. Dev A runs them after every prompt change; target ⩾ 8 of 10 cite the expected moment.

8. Risks, guardrails and open questions

The biggest risk is a confident answer built on an outdated or misheard source; grounding, dates and "not covered" handling are the defenses.

Risk	Guardrail
Hallucinated advice	Answer only from retrieved segments; every sentence needs a citation; "not covered" path is mandatory
Stale policy (testing, deadlines, essay prompts change yearly)	Show publish date on every citation; flag sources older than 18 months; tell user to confirm on the official admissions site
Auto-caption errors on names and numbers	Show the raw transcript snippet next to the summary so the user can verify by playing the clip
Officer's words vs model's synthesis blurred	Label key quotes as verbatim; label answers as "summary of sources"
Content terms of use	Personal research use only; store transcripts privately; no public republishing of transcripts or audio
One-day timeline slips	Cut order in section 4; podcast audio moved to Day 2
Answers miss the right moment	Run the golden test set after every change; tune segment size, filters and retrieval before rewording prompts

Open questions for AJ (answer before or during Hour 1):

- [ ] Hosting: run locally on a laptop, or on one cloud VM so Aksh can use it too?

- [ ] Which single question should be the Day-1 demo (e.g., "How should I answer Yale's why-this-major question?")?
- [ ] AJ fills yale.yaml and mit.yaml with 5–10 URLs each before Hour 1 (e.g., the Yale admissions podcast episodes, key admissions blog pages, essay-prompt page.)
- [ ] Segment length: keep 60 s minute summaries, or rely on chapters plus 2-minute segments to reduce noise?
- [ ] Should Aksh get his own login and Notes, or share AJ's?
- [ ] Do you want the 10-question golden test set from you by Day 1 morning?